ULTIMATE TERMINAL

Project Feature List & Development Roadmap

Product vision	A lightweight, cross-platform developer workspace combining a minimal terminal UI, flexible panels, and a persistent Go-powered backend.

1. Core Features

Terminal Engine | P0 · Essential

- Native shell support: PowerShell, CMD, Bash, Zsh, and other installed shells.
- Independent persistent terminal sessions, tabs, split panes, and resizable terminals.
- ANSI colours, Unicode, interactive TUI applications, and scrollback.
- Copy, paste, search, clear, resize, and configurable terminal fonts.
- Compatibility with Claude Code, Git, Go, npm, and other CLI tools.
- Session lifecycle management and process cleanup.

Workspace and Panel System | P0 · Essential

- Multiple tabs and nested split layouts.
- Drag-and-drop tab reordering and panel splitting.
- Resize, move, rename, duplicate, and close panels.
- Save and restore named workspace layouts.
- Restore layout metadata without reviving stale processes.
- Support independent panels with their own lifecycle.

Minimal UI and Navigation | P0 · Essential

- Compact title bar and unobtrusive status bar.
- Dark theme with a restrained mint accent.
- Command palette, quick project switcher, and configurable keyboard shortcuts.
- Context sidebar hidden by default.
- Keyboard-first navigation and focus management.

2. Developer Workspace

Project Management | P1

- Named projects and workspaces, recent projects, and pinned favourites.
- Project-specific layouts and working directories.
- Optional new-window support.

File Explorer and Previews | P1

- Browse project files and folders; open files in the default editor.
- Built-in Markdown and image previews, file search, and quick navigation.

- Restrict filesystem access to explicitly permitted paths.

Git Integration | P2

- Current branch and working-tree status.
- Changed, staged, and untracked files.
- Unified or side-by-side diffs, commit history, and branch switching.
- Stage, unstage, commit, pull, and push.

Browser and Development Preview | P2

- Open local development servers inside a panel.
- Preview web applications alongside the terminal.
- Reload, resize, open previews externally, and configure local preview URLs.

3. Remote Development

SSH and SFTP | P3

- Saved SSH profiles, host aliases, key authentication, and supported SSH agents.
- Remote terminal sessions and SFTP browsing/transfers.
- Jump hosts, SSH port forwarding, and reconnect after temporary connection loss.
- Secure credential storage using platform-appropriate facilities.
- Clear distinction between local and remote sessions.

4. AI-Assisted Development

Optional AI Panel | P4

- Configurable providers and user-supplied API keys.
- Explain terminal errors and command output.
- Ask questions about selected files or project context.
- Generate shell commands with explanations and require confirmation before execution.
- Summarise Git diffs and help investigate failures.
- Run Claude Code and other CLI agents independently inside terminal sessions.

Principle: the core application must work fully without an AI subscription or API key.

5. Personalisation and Productivity

- Dark theme first, with light theme later.
- Custom fonts, font sizes, cursor styles, colours, and keyboard shortcuts.
- Searchable command history and workspace templates.
- Task runner for common development commands.
- Desktop notifications for completed tasks and long-running processes.
- Session logs with configurable retention.
- Import/export workspace configurations.
- Optional plugins or a panel extension API.

6. Technical Architecture

Area	Technology / Requirement
Desktop shell	Tauri 2
Frontend	React, TypeScript, Vite
Styling	Tailwind CSS
Terminal renderer	xterm.js
Backend	Persistent Go sidecar
Native bridge	Minimal Rust host
Local database	SQLite
UI state	Zustand
Terminal processes	Windows ConPTY; Unix PTYs on macOS/Linux
IPC	Named pipes on Windows; Unix-domain sockets on macOS/Linux
Platforms	Windows, macOS, and Linux
Distribution	Bundle the Go backend; users should not need Go installed
Security	No unauthenticated TCP listener; restrict IPC and filesystem access

7. Development Roadmap

P0 Usable Terminal

- Launch directly into a working terminal
- Tabs, split panes, and independent sessions
- ANSI, Unicode, copy/paste, and resizing
- Claude Code and interactive CLI compatibility
- Keyboard shortcuts and command palette
- Process cleanup and cross-platform smoke tests

P1 Workspace Foundation

- Resizable panel tree and draggable tabs
- Save and restore layouts
- Project switcher and recent projects
- Context sidebar and file explorer
- Theme and terminal preferences

P2 Developer Tools

- Git status and diff panels
- Markdown and image previews
- Embedded local browser preview
- Searchable command history
- Workspace templates and task runner

P3 Remote Development

- SSH profiles and key authentication
- Remote terminal sessions
- SFTP file browsing and transfer
- Port forwarding and jump hosts
- Reconnect and connection status

P4 Extensibility

- Optional AI assistant and provider configuration
- Plugin and panel API
- Notifications and session logs
- Advanced customisation

8. Explicitly Out of Scope for the First Release

- Building or hosting an AI model inside the app.
- A full IDE or code editor.
- A custom terminal rendering engine.
- Cloud account synchronisation.
- A plugin marketplace.
- A complete remote development environment comparable to VS Code Remote.
- Team collaboration and multi-user workspaces.

First-release success criterion	A beautiful, fast terminal with reliable sessions and flexible split workspaces on Windows, macOS, and Linux. All other features should build on this foundation.